

SIF Precursor Detection

Oil India Limited — SIH 2026 | PS 26165

GOARC Industrial Safety 4.0 · Safety 1.0 Prototype

Project Brief — Technologies · Workflow · Problem & Solution · Database · Scalability · Privacy & Hypotheticals · Demo Q&A;

React 19 + Vite · FastAPI + SQLAlchemy · sklearn TF-IDF · PostgreSQL 16 / SQLite · Docker Compose · 5173 / 8000 / 8001 / 5432

yugantyadav/OIL-SIF-PRECURSOR · Proprietary — SIH 2026 · 30 Aug 2026

1 Problem Statement

PS 26165 — Oil India Limited: Every drilling, pipeline, and refinery shift files Unsafe Acts, Unsafe Conditions, and Near-Misses. >80% of Serious Injuries & Fatalities (SIFs) are preceded by such weak signals, but they are buried in free-text narratives and reviewed manually. Result: delayed corrective action, repeat incidents, and regulatory risk. The ask: an AI system that **ingests routine safety reports, flags SIF precursors, maps them to the 9 IOGP Life-Saving Rules, and gives actionable insight before a fatality occurs.**

Why it matters: A single SIF costs lives and shuts an OIL asset for weeks. Early triage (minutes vs weeks) turns a “wet floor near workshop” into a closed hazard before it becomes a fall or hydrocarbon leak into a fire/explosion. Our demo (52 reports, 15 sites) mirrors OIL’s real HSE volume and shows the pipeline working end-to-end.

2 Technologies by Group (incl. Database)

Group	Stack	What They Built	Key Files
Frontend ×2	React 19, Vite 8.2, React Router 7, Recharts 3.10, Poppins 400/600/700, GOARC #FCAB04/#2C52 DB/#1A1E22	4 routes + GOARC hero (48px/700, wave 85px), 4-col stats, side-by-side charts, Reports CRUD + CSV bulk (5 MB/5000-row guard), Details → Analyze locked prefill (5000-char counter)	pages/*, components/*, App.css, index.css, api.js, data/reportsData.js
Backend ×2	FastAPI 0.110, Pydantic 2, SQLAlchemy 2, Uvicorn, httpx, dateutil	8 endpoints, schemas + crud (IntegrityError retry, ilike on all fields), utils (10 date formats + dateutil), lifespan create_all	main.py, crud.py, schemas.py, utils.py, requirements.txt
Database ×1	PostgreSQL 16-alpine, SQLite, SQLAlchemy, Alembic (ready), SQL	8 tables (reports, classifications, lsr_tags, entities, clusters, report_clusters, batches, users), init.sql (UUID/JSONB), database.py StaticPool SQLite vs pool_pre_ping Postgres, indexes idx_reports_site/date, idempotent seed.py --force	database/init.sql, backend/database.py, models.py, seed.py
AI ×1	scikit-learn 1.9, joblib, pandas, numpy, TF-IDF	Hybrid ML (sif_model.pkl + vectorizer.pkl) + 7 lexicon rules (Hydrocarbon Release 0.82 boost), lazy_ensure_loaded(), Docker path fix	ai/main.py, analyser.py, train_model.py, sif_model.pkl
Docker / DevOps	Docker, Compose, python:3.11-slim, node:20-alpine	3 Dockerfiles (EXPOSE 5173, libgomp1), compose 5173:5173 --host, healthchecks, anonymous /app/.venv volumes, .dockerignore, start.sh 3.12 wait-for-health	docker-compose.yml, Dockerfiles, .dockerignore, start.sh

Group	Stack	What They Built	Key Files
Git	Git, GitHub, PAT, rebase	feat/* → develop → main PRs, .env gitignored, 15+ pushes yugantyadav/OIL-SIF-PRECURSOR, GOARC + splash + guards	.gitignore, README.md, docs/*

GOARC: Dark #1A1E22 hero + amber #FCAB04 CTAs, 44px inputs 6px radius, wave dividers. **OIL splash:** 56×56 #FCAB04, 700ms in → 1.4s hold → 650ms fade (sessionStorage). **Database:** 8-table normalized schema, dual-engine (Postgres JSONB vs SQLite Text JSON), idempotent seed from frontend data.

2.1 Database — Deep Dive

Schema (database/init.sql + backend/models.py): **reports** (id PK String36, report_id UNIQUE R-001, report_date Date, site→location, activity, report_type→category, risk, status, reported_by, narrative Text, batch_id, created_at) → **classifications** (sif_probability, sif_flag, confidence, model_version hybrid-v1, explanation_snippets Text JSON) → **lsr_tags** (rule_name, confidence, matched_keywords) → **entities** (activity/location/barrier_failure) → **clusters/report_clusters** (pattern mining ready) → **batches** (filename, total, sif_count) → **users** (email, role). **Indexes:** idx_reports_site/date/batch, idx_classifications_sif_flag, idx_lsr_rule.

Engine: **backend/database.py** reads **DATABASE_URL** (SQLite sqlite:///./sif.db via start.sh vs PostgreSQL postgresql://sif_user@sif_pass@postgres:5432/sif_db via Compose). SQLite: **StaticPool + check_same_thread False** (thread-safe for FastAPI); Postgres: **pool_pre_ping**. **init.sql** (uuid-oss, UUID, JSONB) runs only on first **docker compose** Postgres boot; local uses **Base.metadata.create_all()**. Divergence handled by String(36)+Text JSON in models.

Seed & ops: **backend/seed.py** loads **frontend/src/data/reportsData.js** (50 rows) via regex, **idempotent:** inserts only missing report_id unless **--force** (delete+re-seed). Date parsing **utils.parse_date** 10 formats + **dateutil** fallback (30 Aug → 30 August 2026 OK). **utils.next_report_id** with **IntegrityError** **retry (5×)** avoids race on concurrent R-xxx. Alembic listed in requirements, migrations ready for prod.

3 Workflow — Input to Output

Architecture: Frontend 5173 → Backend 8000 → AI 8001 → DB 5432/SQLite. **VITE_API_URL=http://localhost:8000** (NEXT_PUBLIC alias), fallback **localStorage sif_reports**.

1. **Ingest. + Add Report** (manual) → **POST /api/reports** (IntegrityError retry) or **Bulk CSV** (flexible headers, only description required, 5 MB/5000-row guard) → **POST /api/reports/upload** → bulk_create_reports dedup).
2. **Store.** Parses dates (10 formats + dateutil), maps category→report_type etc., writes **reports + batches**. **seed.py** ensures 50 rows.
3. **Analyze:** Details → **Analyze with AI** (location.state prefill, readOnly locked, 5000-char counter).
4. **AI:** **POST /api/analyze {narrative, report_id}** → httpx → AI **/analyze** (lazy sif_model.pkl, TF-IDF → 0.82 leakage boost + 7 LSR rules → SIF+LSR+entities).
5. **Persist & recall:** save_ai_result → classifications/lsr_tags/entities; **GET /api/reports** (ilike all fields) + **GET /api/dashboard/stats** (15s poll).

4 Problem Statement & Unique Solution

Problem PS 26165: SIF precursors buried in free text → manual review misses 5-10% high-risk → fatalities. Need AI triage + 9 IOGP rules + actionability.

Unique: OIL-tuned hybrid (TF-IDF + Hydrocarbon Release 82%), locked Report→AI, 5 MB/5000-row CSV guard + template, GOARC Safety 4.0 theme + Ferrari OIL splash (56×56), run anywhere **.start.sh** vs **docker compose** with dual DB parity.

5 Scalability & Performance

Today's bottleneck: Single uvicorn, sync **analyze-batch** (500 rows ≈4 min). Limit 1000 no cursor, no Redis. SQLite StaticPool fine for demo.

To 100k/1M: LB + Postgres replicas + idx_reports_site/date + Redis cache for /stats + Celery workers + CDN + GPU AI + cursor pagination + partitioned reports. Guards (5000-char/row) keep ingestion bounded.

6 Privacy, Security & Hypotheticals

Privacy/security: On-prem inference (no external LLM), .env gitignored, ORM (no SQL injection), CORS *→future JWT/RBAC + HTTPS, 5 MB/5000-row limits, health checks only status.

Hypotheticals: AI down → fallback low-prob + warning; DB down → 503 + localStorage offline; Huge CSV → client block + flexible headers (utf-8-sig→latin-1) + auto-rename R-xxx; Overlong/HTML → 5000-char block + Pydantic; Port 8000 busy (Flask) → killed + uvicorn 3.12; Cold start race → lazy load guarded.

7 Demo-Day Judge Q&A; — Likely Asks

Q. Why should oil leakage be SIF? Show it.

Was 57% low → now **High 82% Hydrocarbon Release** (oil leakage/oil leak/leakage boost) + actions *Isolate, cordon, spill containment*. Try *Oil leakage detected...* → Analyze.

Q. Show the locked Report→AI flow.

Reports → R-006 Analyze with AI → /analyze with **readOnly** + 46/5000 + Edit manually toggle. Only Analyze clickable — audit trail.

Q. What if I paste 20k chars or 50k-row CSV?

Blocked: 5000-char + 5 MB/5000-row guard → *Split file*. Backend limit 1000 + 30s queue (future Celery).

Q. How does the chart update after upload?

Polls 15s + visibilitychange/storage + refreshReports() after upload. Fallback localStorage.

Q. SQLite vs Postgres — different data?

No. String(36)+Text JSON + StaticPool vs pool_pre_ping + seed --force idempotent → 50 rows both; init.sql only on first Postgres boot.

Q. What's still mock?

Transformer fine-tuning — still TF-IDF hybrid; clustering ready not in UI; auth next.

Stack: React 19 + Vite + Recharts + FastAPI + SQLAlchemy + sklearn 1.9 + PostgreSQL/SQLite + Docker · 5173/8000/8001/5432 · GOARC #FCAB04 — OIL Safety 1.0 Prototype

yugantyadav/OIL-SIF-PRECURSOR · 30 Aug 2026 · Brief v3 — with Database